

Iterative Methods for Positive Definite and Positive Semidefinite Linear Systems

Regularization, Residual Correction, and Iterative Refinement

Biajid Chowdhury

August 13, 2026

Abstract

Large systems of linear equations arise naturally in numerical computation, particularly after the discretization of partial differential equations. Although direct methods such as Gaussian elimination, LU factorization, QR factorization, and Cholesky factorization provide powerful techniques for solving linear systems, their computational and memory requirements may become substantial for very large problems. This motivates the use of iterative methods, in which a sequence of increasingly accurate approximations is constructed rather than attempting to obtain the solution through a single direct factorization.

This note develops the basic philosophy of iterative solution methods through a regularized iterative refinement scheme for symmetric positive definite and positive semidefinite matrices. Beginning with the linear system

$$Ax = b,$$

we introduce the residual, derive a correction equation, and construct the iteration

$$x_{k+1} = x_k + (A + \epsilon I)^{-1}(b - Ax_k).$$

Particular attention is given to understanding why the method converges, how the eigenvalues of A determine the convergence behavior, what happens when A is positive semidefinite and singular, and how the choice of the regularization parameter ϵ produces a tradeoff between numerical conditioning and convergence speed.

The discussion is developed gradually from the underlying linear algebra, with the objective of making each step of the method computationally and geometrically understandable before presenting its MATLAB implementation.

Contents

1	Introduction	4
1.1	The Basic Idea of an Iterative Method	5
1.2	Different Iterative Schemes for Linear Systems	6
1.3	Positive Definite and Positive Semidefinite Matrices	6
1.4	A Regularized Approach to Iterative Refinement	7

2	Derivation of the Regularized Iterative Refinement Scheme	9
2.1	Regularizing the Original System	9
2.2	Constructing an Initial Approximation	10
2.3	Measuring the Error in the Original Equation	11
2.4	What Should the Correction Satisfy?	12
2.5	Approximating the Correction	13
2.6	From One Correction to an Iterative Process	14
2.7	Why the Inverse Should Not Be Computed Explicitly	15
3	Convergence of the Regularized Iterative Scheme	16
3.1	Deriving the Residual Recurrence	17
3.2	Simplifying the Residual Iteration	18
3.3	Repeated Application of the Residual Matrix	18
3.4	Eigenvalues of the Residual Iteration Matrix	19
3.5	What Happens to an Individual Residual Component?	20
3.6	The Positive Definite Case	20
3.7	A Simple Numerical Interpretation	21
3.8	The Positive Semidefinite Case	22
3.9	Why Consistency Resolves the Singular Case	23
3.10	What if the Singular System Is Inconsistent?	24
3.11	Convergence in Terms of the Spectral Radius	25
3.12	Summary of the Convergence Argument	26
4	The Role of the Regularization Parameter ϵ	27
4.1	How Regularization Changes the Eigenvalues	28
4.2	Effect of ϵ on Conditioning	28
4.3	A Numerical Example of Improved Conditioning	29
4.4	The Singular Positive Semidefinite Case	30
4.5	Effect of ϵ on Convergence	31
4.6	Small ϵ and Fast Convergence	31
4.7	Large ϵ and Slow Convergence	32
4.8	The Conditioning–Convergence Tradeoff	32
4.9	Which Eigenvalue Controls the Slowest Convergence?	33
4.10	The Corresponding Factor for a Singular Matrix	33
4.11	Why Not Choose an Extremely Small ϵ ?	34
4.12	Why Not Choose an Extremely Large ϵ ?	35
4.13	A Practical Principle for Choosing ϵ	35
4.14	How This Principle Will Be Used in MATLAB	35
4.15	Summary	36
5	Computational Algorithm	37
5.1	Step 1: Form the Regularized Matrix	37
5.2	Step 2: Compute the Cholesky Factorization	38
5.3	Step 3: Compute the Initial Approximation	38
5.4	Step 4: Compute the Initial Residual	38

5.5	Step 5: Test for Convergence	39
5.6	Step 6: Compute the Correction	39
5.7	Step 7: Update the Approximation	40
5.8	Pseudocode	40
5.9	Automatic Adjustment of ϵ	40
5.10	Cost per Iteration	41
5.11	A More Compact Computational View	41
5.12	Preparation for MATLAB Implementation	42
6	MATLAB Implementation	42
6.1	MATLAB Function	42
6.2	Connection Between the Code and the Mathematics	46
6.3	Why the Cholesky Factorization Is Reused	47
6.4	Automatic Adjustment of the Regularization Parameter	47
6.5	Stopping Criterion	48
6.6	A Simple Test Example	49
6.7	A Singular Positive Semidefinite Example	50
6.8	Final Computational Interpretation	51
7	Conclusion	52

1 Introduction

A large portion of numerical mathematics eventually leads to the solution of a linear system of the form

$$Ax = b,$$

where

$$A \in \mathbb{R}^{n \times n}, \quad x, b \in \mathbb{R}^n.$$

At first sight, this may appear to be one of the simplest problems in linear algebra. If the matrix A is nonsingular, then formally

$$x = A^{-1}b.$$

In numerical computation, however, we almost never form A^{-1} explicitly. Instead, we solve the linear system by using an appropriate numerical algorithm.

Broadly speaking, numerical methods for solving

$$Ax = b$$

may be divided into two major classes:

Direct Methods	and	Iterative Methods.
----------------	-----	--------------------

Direct methods attempt to obtain the solution after a finite sequence of algebraic operations. Familiar examples include Gaussian elimination, LU factorization, QR factorization, and, for symmetric positive definite matrices, Cholesky factorization.

For matrices of moderate size, these methods are extremely powerful and often provide the most convenient way of solving a linear system.

The situation changes considerably when the matrix becomes very large.

This occurs frequently in the numerical solution of partial differential equations. A differential equation is first replaced by a finite collection of algebraic equations using a discretization technique such as finite differences, finite elements, or finite volumes. After discretization, the original differential equation is transformed into a matrix equation

$$Ax = b.$$

If the underlying spatial grid contains a very large number of points, then the dimension of A may also become extremely large.

Fortunately, such matrices are often sparse; that is, most of their entries are zero. Nevertheless, even when sparsity is exploited, a direct factorization may require considerable memory and computational work. During factorization, additional nonzero entries may also appear in locations that were originally zero. This phenomenon is commonly called *fill-in*.

Consequently, for many large-scale problems arising from partial differential equations, iterative methods become particularly attractive.

It should be emphasized that direct methods are not inherently unsuitable for partial differential equations. Sparse direct solvers remain extremely important in scientific computing. Rather, as the size of the problem increases, memory requirements and computational cost often make iterative methods a more practical alternative.

1.1 The Basic Idea of an Iterative Method

An iterative method does not attempt to obtain the final solution in a single factorization process. Instead, it begins with an approximation

$$x_0$$

and generates a sequence

$$x_0, \quad x_1, \quad x_2, \quad x_3, \quad \dots$$

with the hope that

$$x_k \longrightarrow x^*,$$

where x^* satisfies

$$Ax^* = b.$$

Therefore, the central philosophy of an iterative method is

Start with an approximation, determine how wrong it is, correct it, and repeat.

The phrase “determine how wrong it is” has a precise mathematical meaning. Suppose x_k is the current approximation. If x_k were the exact solution, then

$$Ax_k = b.$$

In general, however,

$$Ax_k \neq b.$$

The difference

$$r_k = b - Ax_k$$

is called the *residual*.

The residual measures how well the current approximation satisfies the original system. If

$$r_k = 0,$$

then

$$Ax_k = b,$$

and therefore x_k is an exact solution.

If $r_k \neq 0$, then the residual contains information that can be used to construct a correction to the current approximation.

This simple observation lies behind a large number of iterative techniques.

1.2 Different Iterative Schemes for Linear Systems

There are many different iterative methods for solving linear systems.

Among the classical stationary methods are

- Jacobi iteration,
- Gauss–Seidel iteration,
- Successive Over-Relaxation (SOR).

More sophisticated methods include Krylov subspace techniques such as

- Conjugate Gradient (CG),
- MINRES,
- GMRES,
- BiCGSTAB.

Different iterative methods are appropriate for different classes of matrices.

For example, the Conjugate Gradient method is particularly important for large symmetric positive definite systems, while GMRES is commonly used for nonsymmetric systems.

Thus, before choosing an iterative method, one should always ask

What mathematical structure does the matrix A possess?

The answer may determine which algorithm is appropriate.

1.3 Positive Definite and Positive Semidefinite Matrices

A particularly important class of matrices appearing in numerical analysis consists of symmetric positive definite and symmetric positive semidefinite matrices.

A symmetric matrix A is positive definite if

$$x^T Ax > 0$$

for every nonzero vector x .

Equivalently, all eigenvalues of A are positive:

$$\lambda_i > 0.$$

A symmetric matrix A is positive semidefinite if

$$x^T A x \geq 0$$

for every vector x .

In this case,

$$\lambda_i \geq 0.$$

Therefore, a positive semidefinite matrix may contain one or more zero eigenvalues and may consequently be singular.

These matrices occur naturally in the discretization of many partial differential equations.

For example, consider the Poisson equation

$$-\nabla^2 u = f.$$

Under suitable Dirichlet boundary conditions, its standard discretization typically produces a symmetric positive definite matrix.

Under pure Neumann boundary conditions, however, the corresponding discrete operator may possess a nontrivial nullspace. In such a case, the matrix is symmetric positive semidefinite and singular.

However, singularity does not automatically imply that a numerical model is incorrect. It may simply reflect a genuine structural property of the underlying differential equation.

1.4 A Regularized Approach to Iterative Refinement

The principal purpose of this note is to study a particularly simple and instructive iterative scheme for symmetric positive definite and positive semidefinite systems.

Suppose that we wish to solve

$$Ax = b,$$

but A is ill-conditioned or possibly singular.

Rather than solving directly with A , we introduce a small positive parameter

$$\epsilon > 0$$

and form the regularized matrix

$$\boxed{A + \epsilon I}.$$

The matrix $A + \epsilon I$ is easier to work with because the addition of ϵI shifts the eigenvalues of A . If

$$Av_i = \lambda_i v_i,$$

then

$$(A + \epsilon I)v_i = (\lambda_i + \epsilon)v_i.$$

Therefore,

$$\boxed{\lambda_i(A + \epsilon I) = \lambda_i(A) + \epsilon.}$$

For a positive semidefinite matrix,

$$\lambda_i \geq 0,$$

and hence

$$\lambda_i + \epsilon > 0.$$

Therefore,

$$A + \epsilon I$$

is symmetric positive definite.

This allows us to compute a Cholesky factorization

$$\boxed{A + \epsilon I = LL^T.}$$

The regularized system can then be used to obtain an initial approximation and subsequently to compute corrections to that approximation.

The resulting iterative scheme is

$$\boxed{x_{k+1} = x_k + (A + \epsilon I)^{-1}(b - Ax_k).}$$

The inverse appearing in this expression is mathematical notation only. In an actual numerical implementation, the inverse should not be formed explicitly. Instead, systems involving $A + \epsilon I$ will be solved using its Cholesky factorization.

The purpose of the following sections is not merely to state this formula, but to develop its mathematical model carefully from

$$Ax = b,$$

understand the role of the residual, prove why the residual converges to zero under the appropriate assumptions, investigate the special case of a singular positive semidefinite matrix, and study the important tradeoff involved in choosing ϵ .

Finally, the complete mathematical procedure will be translated directly into MATLAB code so that every important line of the implementation can be connected to the mathematical development that precedes it.

2 Derivation of the Regularized Iterative Refinement Scheme

We now develop the iterative scheme introduced in the previous section. Rather than beginning with the final iteration formula, we shall derive it directly from the original linear system

$$Ax = b.$$

Throughout this discussion, we assume that A is symmetric positive definite or symmetric positive semidefinite. In the latter case, A may be singular, and whenever A is singular we assume that the system

$$Ax = b$$

is consistent.

The basic idea is simple. We shall first replace the potentially ill-conditioned or singular matrix A by a nearby matrix that is easier to work with. This modified system will provide an initial approximation. We shall then return to the *original* system, measure how well the approximation satisfies it, and use that information to construct a correction.

2.1 Regularizing the Original System

Suppose that

$$A \succeq 0.$$

Since A is symmetric positive semidefinite, all of its eigenvalues are nonnegative:

$$\lambda_i \geq 0.$$

If one or more eigenvalues are very close to zero, then A may be severely ill-conditioned. If an eigenvalue is exactly zero, then A is singular.

We therefore introduce a small parameter

$$\epsilon > 0$$

and replace A temporarily by

$$\boxed{A_\epsilon = A + \epsilon I.}$$

To understand why this helps, let v_i be an eigenvector of A corresponding to the eigenvalue λ_i . Thus,

$$Av_i = \lambda_i v_i.$$

Then

$$(A + \epsilon I)v_i = Av_i + \epsilon I v_i.$$

Since

$$Av_i = \lambda_i v_i$$

and

$$Iv_i = v_i,$$

we obtain

$$(A + \epsilon I)v_i = \lambda_i v_i + \epsilon v_i,$$

and hence

$$\boxed{(A + \epsilon I)v_i = (\lambda_i + \epsilon)v_i.}$$

Therefore, the eigenvalues of the regularized matrix are

$$\boxed{\lambda_i(A + \epsilon I) = \lambda_i(A) + \epsilon.}$$

Since

$$\lambda_i(A) \geq 0$$

and

$$\epsilon > 0,$$

we have

$$\lambda_i(A + \epsilon I) > 0.$$

Consequently,

$$\boxed{A + \epsilon I \succ 0.}$$

Thus, even when A is positive semidefinite and singular, $A + \epsilon I$ is positive definite.

Because the regularized matrix is symmetric positive definite, it admits a Cholesky factorization

$$\boxed{A + \epsilon I = LL^T.}$$

This factorization will later become particularly useful because it can be computed once and reused throughout the entire iterative process.

2.2 Constructing an Initial Approximation

Instead of attempting to solve

$$Ax = b$$

directly, we first solve the regularized system

$$(A + \epsilon I)x_0 = b.$$

Since $A + \epsilon I$ is positive definite, this system has a unique solution. Formally,

$$x_0 = (A + \epsilon I)^{-1}b.$$

It is important, however, to recognize that x_0 is generally *not* the exact solution of the original system.

Indeed, x_0 satisfies

$$(A + \epsilon I)x_0 = b,$$

whereas the equation that we actually wish to satisfy is

$$Ax = b.$$

These are different systems.

Therefore, the regularization has solved one problem but introduced another: it has given us a numerically convenient solution, but that solution corresponds to a slightly modified matrix.

We shall therefore regard x_0 only as an *initial approximation* to the solution of the original problem.

2.3 Measuring the Error in the Original Equation

Once x_0 has been obtained, we return immediately to the original system

$$Ax = b.$$

If x_0 were already an exact solution, then

$$Ax_0 = b.$$

In general this will not be true.

The discrepancy is measured by the residual

$$r_0 = b - Ax_0.$$

The residual therefore tells us how much of the right-hand side b has not yet been reproduced by our current approximation.

If

$$r_0 = 0,$$

then

$$b - Ax_0 = 0,$$

and consequently

$$Ax_0 = b.$$

In this case, no further work is necessary.
Suppose instead that

$$r_0 \neq 0.$$

Then we would like to determine a correction that can be added to x_0 .

2.4 What Should the Correction Satisfy?

Let the desired correction be denoted by

$$d_0.$$

We construct a new approximation

$$x_1 = x_0 + d_0.$$

The natural question is now:

$$\boxed{\text{What equation should } d_0 \text{ satisfy?}}$$

Ideally, we would like x_1 to satisfy the original system exactly. Therefore,

$$Ax_1 = b.$$

Substituting

$$x_1 = x_0 + d_0$$

gives

$$A(x_0 + d_0) = b.$$

Distributing A ,

$$Ax_0 + Ad_0 = b.$$

Rearranging,

$$Ad_0 = b - Ax_0.$$

But from the definition of the residual,

$$b - Ax_0 = r_0.$$

Therefore, the *exact* correction would satisfy

$$\boxed{Ad_0 = r_0.}$$

If A were well-conditioned and easily invertible, we could solve this equation directly and obtain

$$d_0 = A^{-1}r_0.$$

In fact, in exact arithmetic this correction would immediately produce an exact solution:

$$x_1 = x_0 + A^{-1}r_0.$$

However, this would defeat the entire purpose of the method.

The reason we introduced regularization in the first place was that solving systems involving A may be numerically difficult, or A may even be singular. Therefore, instead of solving

$$Ad_0 = r_0,$$

we again use the regularized matrix.

2.5 Approximating the Correction

We replace the exact correction equation

$$Ad_0 = r_0$$

by the regularized correction equation

$$\boxed{(A + \epsilon I)d_0 = r_0.}$$

Thus,

$$d_0 = (A + \epsilon I)^{-1}r_0.$$

Since

$$r_0 = b - Ax_0,$$

we obtain

$$d_0 = (A + \epsilon I)^{-1}(b - Ax_0).$$

Now recall that

$$x_1 = x_0 + d_0.$$

Substituting the expression for d_0 ,

$$x_1 = x_0 + (A + \epsilon I)^{-1}(b - Ax_0).$$

Therefore,

$$x_1 = x_0 + (A + \epsilon I)^{-1}(b - Ax_0).$$

This produces an improved approximation to the solution of the *original* system.

2.6 From One Correction to an Iterative Process

There is no reason to stop after one correction.

Once x_1 has been obtained, we compute its residual

$$r_1 = b - Ax_1.$$

We then solve another regularized correction equation

$$(A + \epsilon I)d_1 = r_1$$

and update

$$x_2 = x_1 + d_1.$$

The same process may be repeated indefinitely.

At the k -th stage, define

$$r_k = b - Ax_k.$$

The correction d_k is obtained from

$$(A + \epsilon I)d_k = r_k.$$

Therefore,

$$d_k = (A + \epsilon I)^{-1}r_k.$$

The next approximation is

$$x_{k+1} = x_k + d_k.$$

Combining these equations gives

$$x_{k+1} = x_k + (A + \epsilon I)^{-1}r_k.$$

Finally, substituting

$$r_k = b - Ax_k,$$

we arrive at the regularized iterative refinement scheme

$$x_{k+1} = x_k + (A + \epsilon I)^{-1}(b - Ax_k).$$

Thus, the entire method may be summarized by three simple operations:

$$\begin{aligned} r_k &= b - Ax_k, \\ (A + \epsilon I)d_k &= r_k, \\ x_{k+1} &= x_k + d_k. \end{aligned}$$

In words,

Compute the residual $\longrightarrow$ compute a correction $\longrightarrow$ update the solution.
--

This cycle is repeated until the residual becomes sufficiently small.

2.7 Why the Inverse Should Not Be Computed Explicitly

Although the iteration is conveniently written as

$$x_{k+1} = x_k + (A + \epsilon I)^{-1} r_k,$$

the matrix inverse should not be formed explicitly in an actual numerical implementation. Recall that

$$A + \epsilon I = LL^T.$$

To compute the correction d_k , we need to solve

$$(A + \epsilon I)d_k = r_k.$$

Using the Cholesky factorization,

$$LL^T d_k = r_k.$$

Introduce an intermediate vector y_k . First solve

$Ly_k = r_k$

by forward substitution, and then solve

$L^T d_k = y_k$

by backward substitution.

Most importantly, the matrix

$$A + \epsilon I$$

does not change from one refinement step to the next. Therefore, its Cholesky factorization

$$A + \epsilon I = LL^T$$

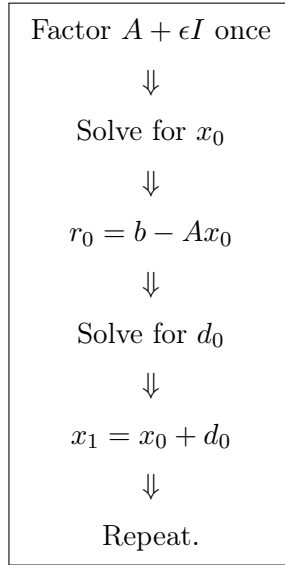
needs to be computed only once.

The same factor L can then be reused to obtain

$$x_0, \quad d_0, \quad d_1, \quad d_2, \quad \dots$$

through inexpensive triangular solves.

This gives the computational structure



We have now derived the iterative scheme, but an important mathematical question remains unanswered:

Why should this iteration converge?

In the next section, we answer this question by deriving an iteration directly for the residual. This will reveal that convergence is governed by the eigenvalues of the matrix

$$\epsilon(A + \epsilon I)^{-1},$$

and ultimately by the quantities

$$\frac{\epsilon}{\lambda_i + \epsilon}.$$

3 Convergence of the Regularized Iterative Scheme

We have derived the iteration

$$x_{k+1} = x_k + (A + \epsilon I)^{-1}(b - Ax_k).$$

The existence of an iterative formula, however, does not by itself guarantee that the generated sequence will approach a solution.

We must therefore answer the fundamental question

Does the iteration actually converge?

Rather than attempting to study x_k directly, it is more convenient to study the residual

$$r_k = b - Ax_k.$$

If

$$r_k \longrightarrow 0,$$

then the approximations increasingly satisfy the original equation

$$Ax = b.$$

We shall therefore derive an iteration for the residual itself and use it to understand the convergence of the method.

3.1 Deriving the Residual Recurrence

Recall that the residual at iteration k is

$$\boxed{r_k = b - Ax_k.}$$

At the next iteration,

$$r_{k+1} = b - Ax_{k+1}.$$

From the iterative scheme,

$$x_{k+1} = x_k + (A + \epsilon I)^{-1} r_k.$$

Substituting this expression into the definition of r_{k+1} ,

$$r_{k+1} = b - A [x_k + (A + \epsilon I)^{-1} r_k].$$

Distributing A ,

$$r_{k+1} = b - Ax_k - A(A + \epsilon I)^{-1} r_k.$$

But

$$b - Ax_k = r_k.$$

Therefore,

$$r_{k+1} = r_k - A(A + \epsilon I)^{-1} r_k.$$

Factoring out r_k ,

$$\boxed{r_{k+1} = [I - A(A + \epsilon I)^{-1}] r_k.}$$

At first sight, this expression does not immediately reveal whether the residual becomes smaller. However, the matrix inside the brackets can be simplified considerably.

3.2 Simplifying the Residual Iteration

Observe that

$$A = (A + \epsilon I) - \epsilon I.$$

Therefore,

$$A(A + \epsilon I)^{-1} = [(A + \epsilon I) - \epsilon I](A + \epsilon I)^{-1}.$$

Distributing,

$$A(A + \epsilon I)^{-1} = (A + \epsilon I)(A + \epsilon I)^{-1} - \epsilon I(A + \epsilon I)^{-1}.$$

Hence,

$$A(A + \epsilon I)^{-1} = I - \epsilon(A + \epsilon I)^{-1}.$$

Substituting this into the residual recurrence gives

$$r_{k+1} = [I - (I - \epsilon(A + \epsilon I)^{-1})] r_k.$$

The two identity matrices cancel, leaving

$$\boxed{r_{k+1} = \epsilon(A + \epsilon I)^{-1} r_k.}$$

This equation is the key to understanding the convergence of the entire method. For convenience, define

$$\boxed{M = \epsilon(A + \epsilon I)^{-1}.}$$

Then

$$\boxed{r_{k+1} = M r_k.}$$

Thus, convergence of the residual is controlled entirely by repeated multiplication by the matrix M .

3.3 Repeated Application of the Residual Matrix

Starting with

$$r_1 = M r_0,$$

the next residual is

$$r_2 = M r_1.$$

Substituting $r_1 = Mr_0$,

$$r_2 = M(Mr_0) = M^2r_0.$$

Similarly,

$$r_3 = Mr_2 = M(M^2r_0) = M^3r_0.$$

Continuing in this manner gives

$$\boxed{r_k = M^k r_0.}$$

Since

$$M = \epsilon(A + \epsilon I)^{-1},$$

we obtain

$$\boxed{r_k = [\epsilon(A + \epsilon I)^{-1}]^k r_0.}$$

Therefore, the question

$$r_k \longrightarrow 0$$

has been transformed into the question

$$M^k r_0 \longrightarrow 0.$$

To understand when this occurs, we now examine the eigenvalues of M .

3.4 Eigenvalues of the Residual Iteration Matrix

Let v_i be an eigenvector of A corresponding to the eigenvalue λ_i :

$$Av_i = \lambda_i v_i.$$

Then

$$(A + \epsilon I)v_i = (\lambda_i + \epsilon)v_i.$$

Consequently,

$$(A + \epsilon I)^{-1}v_i = \frac{1}{\lambda_i + \epsilon}v_i.$$

Multiplying by ϵ ,

$$\epsilon(A + \epsilon I)^{-1}v_i = \frac{\epsilon}{\lambda_i + \epsilon}v_i.$$

But

$$M = \epsilon(A + \epsilon I)^{-1}.$$

Therefore,

$$Mv_i = \frac{\epsilon}{\lambda_i + \epsilon}v_i.$$

Hence the eigenvalues of the residual iteration matrix M are

$$\mu_i = \frac{\epsilon}{\lambda_i + \epsilon}.$$

These quantities are the convergence factors of the method.

3.5 What Happens to an Individual Residual Component?

Because A is symmetric, its eigenvectors may be chosen to form an orthonormal basis. Therefore, the initial residual can be expanded as

$$r_0 = c_1v_1 + c_2v_2 + \cdots + c_nv_n.$$

Applying M once gives

$$r_1 = c_1\mu_1v_1 + c_2\mu_2v_2 + \cdots + c_n\mu_nv_n.$$

After k iterations,

$$r_k = \sum_{i=1}^n c_i \left(\frac{\epsilon}{\lambda_i + \epsilon} \right)^k v_i.$$

This formula makes the convergence mechanism transparent.

Each eigenvector component of the residual evolves independently, and the component associated with λ_i is multiplied at every iteration by

$$\frac{\epsilon}{\lambda_i + \epsilon}.$$

Therefore, convergence can be studied one eigenvalue at a time.

3.6 The Positive Definite Case

Suppose first that A is symmetric positive definite.

Then

$$\lambda_i > 0$$

for every i .

Since

$$\epsilon > 0,$$

we have

$$\lambda_i + \epsilon > \epsilon.$$

Therefore,

$$0 < \frac{\epsilon}{\lambda_i + \epsilon} < 1.$$

Hence,

$$\boxed{0 < \mu_i < 1.}$$

For any number q satisfying

$$0 < q < 1,$$

we know that

$$q^k \longrightarrow 0 \quad \text{as } k \longrightarrow \infty.$$

Therefore,

$$\left(\frac{\epsilon}{\lambda_i + \epsilon} \right)^k \longrightarrow 0.$$

Every eigenvector component of the residual consequently approaches zero, and thus

$$\boxed{r_k \longrightarrow 0.}$$

Therefore, for a symmetric positive definite matrix, the regularized iterative refinement scheme converges for every $\epsilon > 0$.

3.7 A Simple Numerical Interpretation

Suppose, for example, that one eigenvalue of A is

$$\lambda_i = 4$$

and we choose

$$\epsilon = 1.$$

Then the corresponding convergence factor is

$$\frac{\epsilon}{\lambda_i + \epsilon} = \frac{1}{4 + 1} = \frac{1}{5}.$$

Thus, if the corresponding component of the initial residual has magnitude 10, then successive iterations produce approximately

$$10, \quad 2, \quad 0.4, \quad 0.08, \quad 0.016, \quad \dots$$

The component decreases geometrically because

$$\left(\frac{1}{5}\right)^k \rightarrow 0.$$

This illustrates why the quantities

$$\frac{\epsilon}{\lambda_i + \epsilon}$$

may be interpreted as contraction factors for the residual.

3.8 The Positive Semidefinite Case

The positive semidefinite case requires more care.

Suppose

$$A \succeq 0.$$

Then

$$\lambda_i \geq 0.$$

For every strictly positive eigenvalue,

$$\lambda_i > 0,$$

we again have

$$0 < \frac{\epsilon}{\lambda_i + \epsilon} < 1.$$

Therefore, every residual component corresponding to a positive eigenvalue converges toward zero.

However, suppose that

$$\lambda_i = 0.$$

Then

$$\frac{\epsilon}{\lambda_i + \epsilon} = \frac{\epsilon}{\epsilon} = 1.$$

Therefore,

$$\boxed{\mu_i = 1 \quad \text{when } \lambda_i = 0.}$$

This means that a residual component lying in the nullspace of A would *not* shrink.

At first sight, this seems to prevent convergence whenever A is singular.

However, this is precisely where the assumption that the original system is consistent becomes important.

3.9 Why Consistency Resolves the Singular Case

Suppose that A is symmetric positive semidefinite and singular.

Then its nullspace

$$\mathcal{N}(A)$$

is nontrivial.

Because A is symmetric,

$$\boxed{\mathcal{R}(A) = \mathcal{N}(A)^\perp.}$$

Now assume that

$$Ax = b$$

is consistent.

Consistency means that there exists at least one vector x^* such that

$$Ax^* = b.$$

Therefore,

$$b \in \mathcal{R}(A).$$

Hence,

$$b \perp \mathcal{N}(A).$$

In other words, b contains no component in any eigenvector direction associated with

$$\lambda_i = 0.$$

The initial approximation is

$$x_0 = (A + \epsilon I)^{-1}b.$$

Because $A + \epsilon I$ has the same eigenvectors as A , applying $(A + \epsilon I)^{-1}$ to b does not introduce a component in a nullspace direction that was absent from b .

Thus x_0 also has no component in the nullspace of A .

Consequently,

$$Ax_0 \in \mathcal{R}(A),$$

and therefore

$$r_0 = b - Ax_0$$

also belongs to

$$\mathcal{R}(A).$$

Hence,

$$\boxed{r_0 \perp \mathcal{N}(A)}.$$

The troublesome components corresponding to

$$\lambda_i = 0$$

are therefore absent from the residual.

All residual components that are actually present correspond to

$$\lambda_i > 0,$$

and for each of them,

$$0 < \frac{\epsilon}{\lambda_i + \epsilon} < 1.$$

Thus,

$$\boxed{r_k \longrightarrow 0}$$

even though A itself is singular.

3.10 What if the Singular System Is Inconsistent?

The consistency assumption should not be overlooked.

Suppose that A is singular and that b contains a nonzero component in the nullspace of A .

Then $b \notin \mathcal{R}(A)$, and the equation

$$Ax = b$$

has no exact solution.

That nullspace component cannot be produced by Ax , regardless of the choice of x .

Since the convergence factor associated with a zero eigenvalue is

$$\frac{\epsilon}{0 + \epsilon} = 1,$$

that component of the residual remains unchanged.

Therefore, in the inconsistent singular case,

$$r_k$$

cannot converge to zero.

This is not a failure of the numerical algorithm. Rather, it reflects the mathematical fact that no vector x exists for which

$$Ax = b.$$

3.11 Convergence in Terms of the Spectral Radius

The preceding analysis can also be expressed using the spectral radius.

For a matrix M , the spectral radius is defined as

$$\rho(M) = \max_i |\mu_i|,$$

where μ_i are the eigenvalues of M .

For the present iteration,

$$M = \epsilon(A + \epsilon I)^{-1},$$

with eigenvalues

$$\mu_i = \frac{\epsilon}{\lambda_i + \epsilon}.$$

For a positive definite matrix,

$$0 < \mu_i < 1$$

for every i , and therefore

$$\boxed{\rho(M) < 1.}$$

This guarantees

$$M^k \longrightarrow 0,$$

and hence

$$r_k = M^k r_0 \longrightarrow 0.$$

For a singular positive semidefinite matrix,

$$\rho(M) = 1$$

because the zero eigenvalues of A produce eigenvalues equal to one in M .

Therefore, we cannot claim that

$$M^k \longrightarrow 0$$

on the entire vector space.

However, for a consistent system the residual lies entirely in

$$\mathcal{R}(A),$$

where all relevant eigenvalues of M satisfy

$$|\mu_i| < 1.$$

Thus, the iteration is contractive on the subspace in which the residual actually lives.

3.12 Summary of the Convergence Argument

The complete convergence argument can now be summarized as follows.

Starting from

$$x_{k+1} = x_k + (A + \epsilon I)^{-1}(b - Ax_k),$$

we define

$$r_k = b - Ax_k.$$

This leads to

$$r_{k+1} = \epsilon(A + \epsilon I)^{-1}r_k.$$

Therefore,

$$r_k = [\epsilon(A + \epsilon I)^{-1}]^k r_0.$$

If

$$Av_i = \lambda_i v_i,$$

then the corresponding residual component is multiplied at each iteration by

$$\boxed{\frac{\epsilon}{\lambda_i + \epsilon}}.$$

For every positive eigenvalue,

$$\lambda_i > 0,$$

we have

$$0 < \frac{\epsilon}{\lambda_i + \epsilon} < 1,$$

and hence that component converges to zero.

Therefore,

$A \succ 0$	$\implies$	$r_k \rightarrow 0,$
$A \succeq 0$, A singular, and $Ax = b$ consistent	$\implies$	$r_k \rightarrow 0.$

We have therefore established why the regularized iterative refinement scheme converges for the class of problems considered in this note.

There remains, however, an important practical question.

The parameter ϵ appears both in the regularized matrix

$$A + \epsilon I$$

and in the convergence factor

$$\frac{\epsilon}{\lambda_i + \epsilon}.$$

Changing ϵ therefore changes both the numerical conditioning of the system that we solve and the speed at which the iterative refinement converges.

This leads to an important tradeoff, which we examine next.

4 The Role of the Regularization Parameter ϵ

The regularization parameter

$$\epsilon > 0$$

plays two different roles in the iterative scheme.

First, it modifies the matrix

$$A$$

into the regularized matrix

$$A + \epsilon I.$$

This can substantially improve the numerical properties of the system that must be solved at every iteration.

At the same time, ϵ also appears in the convergence factors

$$\frac{\epsilon}{\lambda_i + \epsilon}.$$

Therefore, the choice of ϵ affects both

the conditioning of the regularized system

and

the convergence speed of the iteration.

These two effects work in opposite directions. Understanding this tradeoff is therefore essential.

4.1 How Regularization Changes the Eigenvalues

Suppose that A is symmetric positive semidefinite, with eigenvalues

$$0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n.$$

The eigenvalues of

$$A + \epsilon I$$

are

$$\lambda_i + \epsilon.$$

Thus,

$$\boxed{\lambda_i \longrightarrow \lambda_i + \epsilon.}$$

The regularization therefore shifts the entire spectrum of A to the right by the amount ϵ . For example, suppose

$$\lambda(A) = \{10^{-5}, 2, 10\}.$$

If

$$\epsilon = 0.1,$$

then

$$\lambda(A + \epsilon I) = \{0.10001, 2.1, 10.1\}.$$

The smallest eigenvalue has changed from

$$10^{-5}$$

to

$$0.10001.$$

Although the larger eigenvalues have changed only slightly, the very small eigenvalue has changed dramatically.

This is precisely why regularization can greatly improve the conditioning of a nearly singular matrix.

4.2 Effect of ϵ on Conditioning

For a symmetric positive definite matrix A , the condition number in the Euclidean norm is

$$\kappa_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

If

$$\lambda_{\min} = \lambda_1$$

and

$$\lambda_{\max} = \lambda_n,$$

then

$$\kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

After regularization,

$$A_\epsilon = A + \epsilon I,$$

and therefore

$$\lambda_{\min}(A_\epsilon) = \lambda_{\min} + \epsilon,$$

while

$$\lambda_{\max}(A_\epsilon) = \lambda_{\max} + \epsilon.$$

Consequently,

$$\boxed{\kappa_2(A + \epsilon I) = \frac{\lambda_{\max} + \epsilon}{\lambda_{\min} + \epsilon}.$$

This expression immediately reveals the effect of increasing ϵ .
If $\lambda_{\min}$ is extremely small, then the denominator

$$\lambda_{\min} + \epsilon$$

can increase substantially even for a relatively small value of ϵ .
As a result, the condition number can decrease dramatically.

4.3 A Numerical Example of Improved Conditioning

Consider again a matrix whose extreme eigenvalues are

$$\lambda_{\min} = 10^{-5}$$

and

$$\lambda_{\max} = 10.$$

Before regularization,

$$\kappa_2(A) = \frac{10}{10^{-5}} = 10^6.$$

Thus, the matrix is severely ill-conditioned.
Now choose

$$\epsilon = 0.1.$$

Then

$$\kappa_2(A + \epsilon I) = \frac{10 + 0.1}{10^{-5} + 0.1}.$$

Therefore,

$$\kappa_2(A + \epsilon I) = \frac{10.1}{0.10001} \approx 101.$$

Thus, regularization has changed the condition number approximately from

$$10^6$$

to

$$101.$$

This is an enormous improvement.

We therefore obtain the first important principle:

Increasing ϵ generally improves the conditioning of $A + \epsilon I$.

4.4 The Singular Positive Semidefinite Case

If A is positive semidefinite and singular, then

$$\lambda_{\min}(A) = 0.$$

Therefore,

$$\kappa_2(A) = \infty$$

in the usual sense, since A is singular.

However, for every

$$\epsilon > 0,$$

the smallest eigenvalue of the regularized matrix is

$$\lambda_{\min}(A + \epsilon I) = \epsilon,$$

while the largest eigenvalue is

$$\lambda_{\max}(A + \epsilon I) = \lambda_{\max} + \epsilon.$$

Hence,

$$\kappa_2(A + \epsilon I) = \frac{\lambda_{\max} + \epsilon}{\epsilon}.$$

Thus, the singular matrix A has been replaced by a nonsingular positive definite matrix whose condition number is finite.

As ϵ increases, this condition number decreases.

This makes larger values of ϵ attractive from the point of view of numerical conditioning.

Unfortunately, this is only one side of the story.

4.5 Effect of ϵ on Convergence

From the convergence analysis, the residual component associated with the eigenvalue λ_i is multiplied at every iteration by

$$q_i = \frac{\epsilon}{\lambda_i + \epsilon}.$$

For a positive eigenvalue,

$$\lambda_i > 0,$$

we have

$$0 < q_i < 1.$$

The smaller q_i is, the more rapidly that component of the residual decays.

After k iterations, that component has been multiplied by

$$q_i^k = \left(\frac{\epsilon}{\lambda_i + \epsilon} \right)^k.$$

Therefore, rapid convergence is associated with a small value of

$$\frac{\epsilon}{\lambda_i + \epsilon}.$$

Now examine what happens when ϵ changes.

4.6 Small ϵ and Fast Convergence

Suppose

$$\lambda_i = 1.$$

If

$$\epsilon = 0.01,$$

then

$$q_i = \frac{0.01}{1 + 0.01} = \frac{0.01}{1.01} \approx 0.0099.$$

Thus, after one iteration, the corresponding residual component is only about 0.99% of its previous value.

After two iterations, it is multiplied by approximately

$$(0.0099)^2,$$

which is already extremely small.

Thus, a small value of ϵ can produce very rapid convergence.

4.7 Large ϵ and Slow Convergence

Now keep

$$\lambda_i = 1$$

but choose

$$\epsilon = 10.$$

Then

$$q_i = \frac{10}{1 + 10} = \frac{10}{11} \approx 0.9091.$$

Therefore, each iteration reduces this residual component only according to

$$r \longrightarrow 0.9091r.$$

After two iterations,

$$r \longrightarrow (0.9091)^2 r,$$

and after k iterations,

$$r \longrightarrow (0.9091)^k r.$$

The component still converges toward zero, but the convergence is much slower.

We therefore obtain the second important principle:

Increasing ϵ generally slows the iterative convergence.

4.8 The Conditioning–Convergence Tradeoff

We have now arrived at an important conflict.

From the point of view of conditioning,

$$\epsilon \uparrow \implies \kappa_2(A + \epsilon I) \downarrow.$$

Thus, increasing ϵ generally makes the regularized system easier to solve numerically. However, from the point of view of convergence,

$$\boxed{\epsilon \uparrow \implies \frac{\epsilon}{\lambda_i + \epsilon} \uparrow.}$$

Therefore, increasing ϵ causes the convergence factors to move closer to 1, resulting in slower refinement.

The situation can therefore be summarized as

	Conditioning of $A + \epsilon I$	Iterative Convergence
ϵ small	worse	faster
ϵ large	better	slower

This is the fundamental tradeoff involved in choosing the regularization parameter.

4.9 Which Eigenvalue Controls the Slowest Convergence?

There is another useful observation hidden inside the convergence factor

$$q_i = \frac{\epsilon}{\lambda_i + \epsilon}.$$

For fixed $\epsilon > 0$, this quantity becomes larger as λ_i becomes smaller.

Therefore, among the positive eigenvalues, the smallest positive eigenvalue produces the largest convergence factor and hence the slowest-decaying residual component.

If A is positive definite, this is simply

$$\lambda_{\min}.$$

Hence the slowest convergence factor is

$$\boxed{q_{\max} = \frac{\epsilon}{\lambda_{\min} + \epsilon}.$$

This gives

$$\|r_k\|_2 \leq \left(\frac{\epsilon}{\lambda_{\min} + \epsilon} \right)^k \|r_0\|_2.$$

Thus, when A contains a very small positive eigenvalue, convergence in that eigendirection may be considerably slower than convergence in directions associated with larger eigenvalues.

4.10 The Corresponding Factor for a Singular Matrix

For a singular positive semidefinite matrix,

$$\lambda_{\min} = 0.$$

However, as established in the previous section, the zero-eigenvalue directions do not occur in the residual when the system is consistent.

Therefore, the relevant quantity is not the zero eigenvalue, but the *smallest positive eigenvalue*. Let

$$\lambda_{\min}^+$$

denote the smallest strictly positive eigenvalue of A .

Then, on the range of A , the slowest convergence factor is

$$q_{\max} = \frac{\epsilon}{\lambda_{\min}^+ + \epsilon}.$$

Thus, for a consistent singular positive semidefinite system,

$$\|r_k\|_2 \leq \left(\frac{\epsilon}{\lambda_{\min}^+ + \epsilon} \right)^k \|r_0\|_2.$$

This expression provides a useful quantitative description of the convergence rate in the singular case.

4.11 Why Not Choose an Extremely Small ϵ ?

The convergence analysis may appear to suggest that we should simply choose

$$\epsilon$$

as small as possible.

Indeed,

$$\epsilon \rightarrow 0$$

causes

$$\frac{\epsilon}{\lambda_i + \epsilon} \rightarrow 0$$

for every positive eigenvalue λ_i .

In exact arithmetic, this would appear ideal.

Numerical computation, however, takes place in finite precision.

If A is singular or extremely ill-conditioned and ϵ is chosen too small, then

$$A + \epsilon I$$

may itself remain extremely ill-conditioned.

The Cholesky factorization and subsequent triangular solves may then be sensitive to floating-point roundoff.

Therefore, choosing an extremely small value of ϵ may improve the theoretical convergence factor while simultaneously making the numerical solution of the correction equations less reliable.

4.12 Why Not Choose an Extremely Large ϵ ?

At the opposite extreme, suppose

$$\epsilon \gg \lambda_i.$$

Then

$$\lambda_i + \epsilon \approx \epsilon,$$

and therefore

$$\frac{\epsilon}{\lambda_i + \epsilon} \approx 1.$$

Thus,

$$r_{k+1} \approx r_k.$$

The regularized system may be beautifully conditioned, but each refinement step makes only a small improvement.

Therefore, an excessively large value of ϵ defeats the purpose of the iterative correction.

4.13 A Practical Principle for Choosing ϵ

The ideal value of ϵ therefore lies between two undesirable extremes.

We want ϵ to be

large enough to make $A + \epsilon I$ numerically manageable,

but at the same time

small enough to preserve rapid iterative convergence.

A useful practical philosophy is therefore

Use the smallest regularization that produces a sufficiently stable regularized system.

There is no single value of ϵ that is optimal for every problem. Its appropriate magnitude depends on the spectrum and scaling of A , the working precision, and the accuracy required from the solution.

4.14 How This Principle Will Be Used in MATLAB

In the MATLAB implementation developed later, we will exploit the fact that

$$A + \epsilon I$$

should be symmetric positive definite.

MATLAB's Cholesky routine can return a flag indicating whether the factorization was successful:

$$[L, p] = \text{chol}(A + \epsilon I).$$

Conceptually,

$$p = 0$$

indicates that the Cholesky factorization succeeded, whereas

$$p \neq 0$$

indicates that the matrix was not recognized as positive definite.

This gives us a practical numerical test.

Rather than explicitly computing all eigenvalues merely to determine whether a chosen value of ϵ is adequate, we may attempt the Cholesky factorization directly.

If necessary, ϵ can be increased and the factorization tried again.

For example, one simple strategy is

$$\epsilon, \quad 2\epsilon, \quad 4\epsilon, \quad 8\epsilon, \quad \dots$$

until a successful factorization is obtained.

This avoids a potentially expensive full eigenvalue decomposition and uses the same factorization that will subsequently be required by the iterative algorithm.

Once a suitable ϵ has been obtained and

$$A + \epsilon I = LL^T$$

has been computed, the factor L is stored and reused throughout the refinement process.

4.15 Summary

The regularization parameter ϵ should not be regarded merely as a technical device for avoiding singularity. It directly controls two competing properties of the algorithm.

For conditioning,

$$\epsilon \uparrow \implies A + \epsilon I \text{ generally becomes better conditioned.}$$

For convergence,

$$\epsilon \uparrow \implies \frac{\epsilon}{\lambda_i + \epsilon} \uparrow \implies \text{slower residual decay.}$$

Thus,

$$\text{Better conditioning} \longleftrightarrow \text{slower convergence.}$$

The practical objective is not to make ϵ as large or as small as possible, but rather to find a value that provides an acceptable balance between these two competing effects.

With the derivation, convergence theory, and role of the regularization parameter now established, we are ready to translate the complete procedure into an explicit computational algorithm.

5 Computational Algorithm

The mathematical development of the regularized iterative refinement scheme can now be translated directly into a computational procedure.

Recall that the iteration is based on the three equations

$$r_k = b - Ax_k,$$

$$(A + \epsilon I)d_k = r_k,$$

and

$$x_{k+1} = x_k + d_k.$$

The main computational idea is that the matrix

$$A + \epsilon I$$

does not change from one iteration to the next.

Therefore, rather than repeatedly factorizing this matrix, we factor it only once and reuse the resulting factors throughout the entire iteration.

5.1 Step 1: Form the Regularized Matrix

Given

$$A$$

and a regularization parameter

$$\epsilon > 0,$$

we first form

$$\boxed{B = A + \epsilon I.}$$

Since A is assumed to be symmetric positive semidefinite,

$$B$$

is symmetric positive definite for every sufficiently positive ϵ .

5.2 Step 2: Compute the Cholesky Factorization

Because B is symmetric positive definite, we compute

$$\boxed{B = LL^T.}$$

This factorization is performed only once.

Every subsequent linear system involving B can then be solved using two triangular systems.

For example, to solve

$$Bx = c,$$

we instead solve

$$Ly = c$$

followed by

$$L^T x = y.$$

This avoids both repeated factorization and the explicit formation of

$$B^{-1}.$$

5.3 Step 3: Compute the Initial Approximation

The initial approximation is obtained from

$$Bx_0 = b.$$

Using

$$B = LL^T,$$

we solve

$$Ly = b$$

and then

$$L^T x_0 = y.$$

This gives the first approximation

$$\boxed{x_0 = (A + \epsilon I)^{-1}b.}$$

5.4 Step 4: Compute the Initial Residual

Once x_0 is available, we return to the original system and compute

$$r_0 = b - Ax_0.$$

The residual must always be formed using the original matrix A , not the regularized matrix $A + \epsilon I$.

This distinction is essential.

The purpose of the residual is to measure how well the current approximation satisfies

$$Ax = b,$$

not how well it satisfies the regularized system.

5.5 Step 5: Test for Convergence

A practical iterative method requires a stopping criterion.

A simple choice is to stop when

$$\|r_k\|_2 < \text{tol},$$

where tol is a prescribed tolerance.

Thus, if

$$\|b - Ax_k\|_2$$

is sufficiently small, the current approximation is accepted.

It is also necessary to impose a maximum number of iterations,

$$k_{\max},$$

in order to prevent an infinite loop in the event that convergence is too slow or fails for numerical reasons.

5.6 Step 6: Compute the Correction

If the residual is not yet sufficiently small, we solve

$$Bd_k = r_k.$$

Since

$$B = LL^T,$$

we solve

$$Ly_k = r_k$$

followed by

$$L^T d_k = y_k.$$

Thus, every refinement step requires only two triangular solves.

5.7 Step 7: Update the Approximation

The correction is added to the current approximation:

$$x_{k+1} = x_k + d_k.$$

The new residual is then computed from

$$r_{k+1} = b - Ax_{k+1}.$$

The process is repeated until either

$$\|r_k\|_2 < \text{tol}$$

or the maximum number of iterations is reached.

5.8 Pseudocode

The complete procedure may be summarized by the following pseudocode.

```

Input:  $A, b, \epsilon, \text{tol}, k_{\max}$ 
 $B = A + \epsilon I$ 
Compute  $B = LL^T$ 
Solve  $LL^T x_0 = b$ 
 $r_0 = b - Ax_0$ 

for  $k = 0, 1, 2, \dots, k_{\max} - 1$ 
  if  $\|r_k\|_2 < \text{tol}$ 
    stop
  end
  Solve  $LL^T d_k = r_k$ 
   $x_{k+1} = x_k + d_k$ 
   $r_{k+1} = b - Ax_{k+1}$ 
end

```

5.9 Automatic Adjustment of ϵ

In practice, the initial value of ϵ may be too small for the regularized matrix

$$A + \epsilon I$$

to be recognized numerically as positive definite.

One practical approach is to attempt the Cholesky factorization.

If the factorization fails, then ϵ is increased.

For example,

$$\epsilon \longrightarrow 2\epsilon \longrightarrow 4\epsilon \longrightarrow 8\epsilon \longrightarrow \dots$$

until the Cholesky factorization succeeds.

This strategy has two advantages.

First, it avoids computing a complete eigenvalue decomposition solely to choose ϵ .

Second, it uses the same Cholesky factorization that is required by the iterative method itself.

The practical rule is therefore

Increase ϵ only until $A + \epsilon I$ becomes numerically positive definite.

Once this occurs, the factorization is stored and reused.

5.10 Cost per Iteration

After the Cholesky factorization has been computed, each refinement step requires

- one matrix-vector product

$$Ax_k,$$

- one forward triangular solve,
- one backward triangular solve,
- and a few vector additions and norm evaluations.

Thus, the expensive factorization is not repeated.

This is one of the principal computational attractions of the method.

5.11 A More Compact Computational View

The entire method may be viewed as a repeated residual-correction cycle:

$$x_k \longrightarrow r_k = b - Ax_k \longrightarrow d_k \longrightarrow x_{k+1} = x_k + d_k.$$

The regularized matrix

$$A + \epsilon I$$

acts as the mechanism by which the residual is converted into a correction.

Thus, the method does not repeatedly solve the original system.

Instead, it repeatedly asks:

Given the current residual, what correction should be added?

The answer is approximated through the regularized system

$$(A + \epsilon I)d_k = r_k.$$

5.12 Preparation for MATLAB Implementation

The MATLAB implementation follows the mathematical algorithm almost line by line.

The main computational ingredients are:

- constructing

$$A + \epsilon I,$$

- performing a Cholesky factorization,
- using triangular solves rather than explicitly computing an inverse,
- computing the residual with the original matrix A ,
- iterating until the residual norm satisfies the prescribed tolerance,
- and automatically increasing ϵ if the initial regularization is insufficient for a stable Cholesky factorization.

The complete MATLAB function is developed in the following section.

6 MATLAB Implementation

We now translate the complete regularized iterative refinement procedure into MATLAB.

The implementation follows the mathematical development almost directly.

The main steps are

1. Check the input dimensions and symmetry of A ,
2. Form $A + \epsilon I$,
3. Adjust ϵ if necessary,
4. Compute one Cholesky factorization,
5. Compute the initial approximation,
6. Compute the residual,
7. Repeatedly solve for a correction and update the solution.

6.1 MATLAB Function

The complete MATLAB function is given below.

```
function [x, iter, residual, epsilon] = iterative_refinement(A, b, ...  
    epsilon, tol, max_iter)  
% ITERATIVE_REFINEMENT  
% Solve Ax = b using regularized iterative refinement.  
%  
% Assumption:  
%     A is symmetric positive definite or
```

```

%      symmetric positive semidefinite.
%
% Inputs:
%   A      : coefficient matrix
%   b      : right-hand side vector
%   epsilon : initial regularization parameter
%   tol     : stopping tolerance
%   max_iter : maximum number of refinement iterations
%
% Outputs:
%   x      : approximate solution of  $Ax = b$ 
%   iter   : number of refinement iterations performed
%   residual : final residual norm  $\|b - Ax\|_2$ 
%   epsilon : regularization parameter actually used

[n, m] = size(A);

% Check that A is square
if n ~= m
    error('A must be a square matrix.');
```

end

```

% Check symmetry
if norm(A - A', 'fro') > 1e-12
    error('A must be symmetric.');
```

end

```

% Force b to be a column vector
b = b(:);

% Check dimensions
if length(b) ~= n
    error('Dimension mismatch between A and b.');
```

end

```

% Check epsilon
if epsilon <= 0
    error('epsilon must be positive.');
```

end

```

% -----
% Form regularized matrix
```

```

% -----

B = A + epsilon * eye(n);

% Attempt Cholesky factorization
[L, p] = chol(B, 'lower');

% -----
% Automatically increase epsilon if necessary
% -----

while p ~= 0

    epsilon = 2 * epsilon;

    B = A + epsilon * eye(n);

    [L, p] = chol(B, 'lower');

end

% -----
% Initial approximation
%
%       $(A + \epsilon I)x_0 = b$ 
%
% Since
%
%       $A + \epsilon I = L'L'$ 
%
% solve
%
%       $L*y = b$ 
%       $L'*x = y$ 
% -----

y = L \ b;

x = L' \ y;

% -----

```

```

% Initial residual
% -----

r = b - A * x;

iter = 0;

% -----
% Iterative refinement
% -----

for k = 1:max_iter

    % Check convergence
    if norm(r, 2) < tol
        break;
    end

    % Solve
    %
    %      (A + epsilon*I)d = r
    %
    % using the same Cholesky factorization

    y = L \ r;

    d = L' \ y;

    % Update solution
    x = x + d;

    % Compute new residual using the original matrix A
    r = b - A * x;

    % Update iteration counter
    iter = k;

end

% -----
% Final residual norm
% -----

```

```
residual = norm(r, 2);
```

```
end
```

6.2 Connection Between the Code and the Mathematics

The most important lines of the MATLAB implementation correspond directly to the equations derived earlier.

The regularized matrix

$$A + \epsilon I$$

is formed by

```
B = A + epsilon * eye(n);
```

The Cholesky factorization

$$B = LL^T$$

is computed by

```
[L, p] = chol(B, 'lower');
```

The initial approximation satisfies

$$(A + \epsilon I)x_0 = b.$$

Instead of forming an inverse, MATLAB solves

$$Ly = b$$

and

$$L^T x_0 = y$$

through

```
y = L \ b;  
x = L' \ y;
```

The residual

$$r_k = b - Ax_k$$

is computed using

```
r = b - A * x;
```

The correction satisfies

$$(A + \epsilon I)d_k = r_k.$$

Again using the Cholesky factorization,

```
y = L \ r;  
d = L' \ y;
```

Finally, the update

$$x_{k+1} = x_k + d_k$$

is implemented by

```
x = x + d;
```

Thus, the MATLAB code is not a separate computational construction. It is simply the mathematical algorithm translated directly into numerical operations.

6.3 Why the Cholesky Factorization Is Reused

The matrix

$$A + \epsilon I$$

remains unchanged throughout the refinement process.

Therefore, once

$$A + \epsilon I = LL^T$$

has been computed, there is no reason to factor the matrix again.

Each iteration requires only the triangular systems

$$Ly_k = r_k$$

and

$$L^T d_k = y_k.$$

This is considerably more efficient than repeatedly computing a new factorization.

The computational philosophy is therefore

Factor once, solve many times.

6.4 Automatic Adjustment of the Regularization Parameter

The command

```
[L, p] = chol(B, 'lower');
```

returns a value

$$p = 0$$

when the Cholesky factorization is successful.

If

$$p \neq 0,$$

the matrix is not numerically recognized as positive definite.

In that case, the code increases

$$\epsilon$$

according to

$$\epsilon \leftarrow 2\epsilon.$$

The regularized matrix is then reconstructed and Cholesky factorization is attempted again.

This procedure continues until

$$A + \epsilon I$$

is numerically positive definite.

The corresponding MATLAB code is

```
while p ~= 0

    epsilon = 2 * epsilon;

    B = A + epsilon * eye(n);

    [L, p] = chol(B, 'lower');

end
```

This provides a simple practical mechanism for avoiding an excessively small value of ϵ .

6.5 Stopping Criterion

The iteration terminates when

$$\|b - Ax_k\|_2 < \text{tol.}$$

In MATLAB,

```
if norm(r, 2) < tol
    break;
end
```


This criterion measures whether the current approximation satisfies the original system sufficiently well.

The maximum iteration count

`max_iter`

provides an additional safeguard in case convergence is slower than expected.

6.6 A Simple Test Example

Consider the symmetric positive definite matrix

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$$

and

$$b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

A simple MATLAB test is

```
clear; clc;

A = [4 1;
     1 3];

b = [1;
     2];

epsilon = 0.1;
tol = 1e-10;
max_iter = 100;

[x, iter, residual, epsilon_used] = ...
    iterative_refinement(A, b, epsilon, tol, max_iter);

disp('Approximate solution:')
disp(x)

disp('Number of iterations:')
disp(iter)

disp('Final residual norm:')
disp(residual)

disp('Regularization parameter used:')
```

```
disp(epsilon_used)
```

The computed solution may be compared with MATLAB's direct solution

```
x_exact = A \ b;
```

and the difference can be examined through

```
norm(x - x_exact, 2)
```

6.7 A Singular Positive Semidefinite Example

The method is particularly interesting when A is singular but the system remains consistent.

Consider

$$A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

The eigenvalues of this matrix are

$$0$$

and

$$2.$$

Therefore,

$$A \succeq 0,$$

but A is singular.

Choose

$$b = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Since

$$b \in \mathcal{R}(A),$$

the system

$$Ax = b$$

is consistent.

The MATLAB test is

```
clear; clc;
```

```
A = [ 1 -1;
```

```

        -1  1];

b = [1;
     -1];

epsilon = 1e-2;
tol = 1e-10;
max_iter = 1000;

[x, iter, residual, epsilon_used] = ...
    iterative_refinement(A, b, epsilon, tol, max_iter);

disp('Approximate solution:')
disp(x)

disp('Number of iterations:')
disp(iter)

disp('Final residual norm:')
disp(residual)

```

Because A is singular, the exact solution is not unique.
Indeed,

$$A \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

requires only

$$x_1 - x_2 = 1.$$

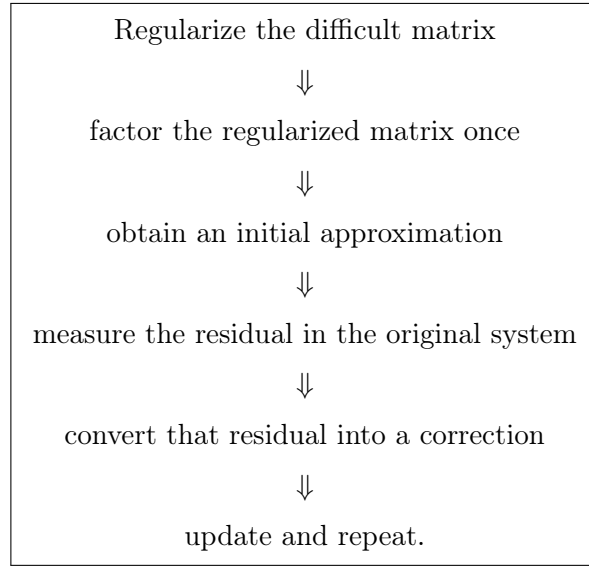
Therefore, infinitely many solutions exist.

The purpose of the iterative method in this case is not necessarily to recover one uniquely defined vector, but rather to obtain a vector whose residual satisfies

$$\boxed{b - Ax \approx 0.}$$

6.8 Final Computational Interpretation

The MATLAB implementation reflects the complete mathematical philosophy of the method:



Thus, the numerical procedure follows the same central idea that motivated the entire discussion:

Approximate, measure, correct, repeat.
--

7 Conclusion

The purpose of this note has been to develop a regularized iterative refinement scheme for solving linear systems

$$Ax = b,$$

with particular attention to symmetric positive definite and positive semidefinite matrices.

We began with a broader observation: there are two major approaches to the numerical solution of linear systems—direct methods and iterative methods. Direct methods such as Gaussian elimination, LU factorization, QR factorization, and Cholesky factorization are among the fundamental tools of numerical linear algebra. For sufficiently large systems, however, particularly those arising from the discretization of partial differential equations, iterative methods may offer substantial computational and memory advantages.

The central philosophy of the iterative method developed in this note can be summarized in a remarkably simple statement:

Approximate, measure, correct, and repeat.
--

Starting from the regularized system

$$(A + \epsilon I)x_0 = b,$$

we obtained an initial approximation x_0 . We then returned to the original problem and measured the discrepancy through the residual

$$r_k = b - Ax_k.$$

By asking what correction would be required to reduce this residual, we derived the regularized correction equation

$$(A + \epsilon I)d_k = r_k$$

and consequently the iteration

$$\boxed{x_{k+1} = x_k + (A + \epsilon I)^{-1}(b - Ax_k).}$$

Rather than accepting this iteration merely as a formula, we examined why it converges. From the residual recurrence

$$r_{k+1} = \epsilon(A + \epsilon I)^{-1}r_k,$$

we obtained

$$r_k = [\epsilon(A + \epsilon I)^{-1}]^k r_0.$$

The eigenvalue analysis then revealed that each residual component associated with an eigenvalue $\lambda_i > 0$ is multiplied at every iteration by

$$\boxed{\frac{\epsilon}{\lambda_i + \epsilon}}.$$

Since

$$0 < \frac{\epsilon}{\lambda_i + \epsilon} < 1,$$

these components decay geometrically.

The singular positive semidefinite case provided an especially useful lesson. When

$$\lambda_i = 0,$$

the corresponding convergence factor becomes 1.

Nevertheless, if the original system is consistent, the residual contains no component in the nullspace of A . Consequently, the components that actually occur in the residual correspond to positive eigenvalues and therefore converge to zero.

The parameter ϵ also illustrated one of the recurring themes of numerical analysis: improving one numerical property may worsen another.

Increasing ϵ improves the conditioning of

$$A + \epsilon I,$$

but at the same time moves the convergence factors

$$\frac{\epsilon}{\lambda_i + \epsilon}$$

closer to 1, thereby slowing the iterative refinement.

Thus,

ϵ too small	$\implies$	potential numerical instability,
ϵ too large	$\implies$	slow convergence.

The objective is therefore not to find a universally “perfect” ϵ , but rather an appropriate compromise for the problem at hand.

The same philosophy extends far beyond the particular algorithm studied here.

In numerical analysis, one should always remember that there is no perfect algorithm for every problem. An algorithm that performs beautifully for one class of matrices may perform poorly, or even fail, for another. At the same time, the computer on which the algorithm is executed does not perform exact mathematics. Floating-point arithmetic introduces rounding error, finite precision, overflow and underflow limitations, and other numerical effects that do not exist in exact algebra.

Consequently, designing a numerical algorithm requires us to consider two imperfect participants simultaneously:

the algorithm	and	the machine.
---------------	-----	--------------

Neither should be trusted blindly.

Perhaps an old family argument provides an appropriately unscientific summary of this very scientific principle. Imagine Grandma declaring from across the dinner table,

“All men are rascals!”

only for Grandpa to reply,

“And no woman is perfect either!”

After several decades of marriage, they have apparently discovered the fundamental theorem of numerical analysis.

The numerical analyst, wisely refusing to take sides, might simply add:

And neither is your algorithm nor your computer.
--

The lesson is not that numerical computation is unreliable. Rather, it is that reliable numerical computation comes from understanding the limitations of the available methods, understanding the structure of the problem, understanding the behavior of finite-precision arithmetic, and then choosing an algorithm accordingly.

There is rarely a universally best method.

There is instead a method that is appropriate for the matrix, the problem size, the available computational resources, and the accuracy that is actually required.

That judgment—knowing not merely *how* an algorithm works, but also *when* and *why* it should be used—is one of the central skills of numerical analysis.

References

- [1] A. Bemporad, *Numerical Optimization: Basics of Numerical Linear Algebra*, IMT School for Advanced Studies Lucca, 2025. Available online: https://cse.lab.imtlucca.it/~bemporad/teaching/numopt/3-basics_linear_algebra.pdf.